2.º DAM / SISTEMAS DE GESTIÓN EMPRESARIAL

# UD1 · EXAMEN DE PYTHON

**CASO PRÁCTICO** · Robo de bicicletas **DURACIÓN:** 120 minutos

**Situación.** BiciAlerta necesita una aplicación de consola para registrar avisos de bicicletas robadas, consultar los casos y actualizar sus datos. Debes completar el proyecto proporcionado, que incluye **tres bicicletas de ejemplo** en **bicicletas.csv**.

## 1. Modelo de datos y organización

Completa los métodos de **Bicicleta** y **RegistroRobos** en **modelos.py**. Cada bicicleta tendrá los siguientes datos:

| Campo | Condiciones |
|---|---|
| **codigo** | Identificador único, no vacío. |
| **propietario / modelo** | Textos no vacíos. |
| **valor** | Número mayor que 0. |
| **estado** | Solo **ROBADA** o **RECUPERADA**. Las altas comienzan en ROBADA. |

El registro dispone de un diccionario cuya clave es el código. Trabaja con **main.py** (menú y validaciones preparados), **modelos.py** (lógica) y **persistencia.py** (CSV).

## 2. Menú de la aplicación

El menú y las validaciones de entrada de **main.py** ya están preparados. Completa la lógica necesaria para que funcionen estas **seis opciones** (además de salir con 0):

| Opción | Funcionalidad requerida |
|---|---|
| **1 · Listar** | Mostrar todas las bicicletas, con código, propietario, modelo, valor y estado. |
| **2 · Registrar robo** | Pedir código, propietario, modelo y valor. Validar los datos e impedir códigos repetidos. |
| **3 · Marcar recuperada** | Buscar por código y cambiar ROBADA a RECUPERADA; no permitir recuperarla dos veces. |
| **4 · Buscar** | Encontrar bicicletas por fragmento de propietario **o** modelo, sin distinguir mayúsculas. |
| **5 · Resumen** | Mostrar total de bicicletas, robadas, recuperadas y suma del valor de las todavía robadas. |
| **6 · Guardar** | Guardar todos los registros y sus estados en **bicicletas.csv**. |
| **0 · Salir** | Finalizar el programa mediante una condición explícita. |

# UD1 · EXAMEN DE PYTHON

## 3. Lectura, guardado y validaciones

- Al iniciar, cargar las bicicletas existentes desde **bicicletas.csv**; al elegir la opción 6, guardarlas en ese mismo fichero. Completa las funciones de **persistencia.py**.
- Utilizar la biblioteca estándar **csv**, **with open(...)**, codificación **UTF-8** y las columnas en este orden: **codigo,propietario,modelo,valor,estado**.
- Convertir el valor del CSV a número, conservar el estado y evitar códigos duplicados. Las validaciones de las entradas de usuario ya están incluidas en **main.py**.
- Las operaciones inválidas no deben modificar el registro. Después de guardar y reiniciar, deben recuperarse correctamente los datos y estados sin duplicados.

## 4. Comprobaciones de funcionamiento

El programa debe superar, entre otras, estas pruebas. Los casos indicados como «CSV inicial» se realizan partiendo de una copia sin cambios del archivo original.

| Prueba | Resultado esperado |
|---|---|
| **Cargar el CSV inicial** | **3** bicicletas: **2** ROBADA y **1** RECUPERADA. Valor pendiente: **540 €**. |
| **Añadir RB04 con valor 180 €** | Total: **4** bicicletas. Valor pendiente: **720 €**. |
| **Intentar añadir RB01 otra vez** | Se rechaza el alta. Ni los registros ni el resumen cambian. |
| **Tras el alta, recuperar RB01** | RB01 pasa a RECUPERADA. Valor pendiente: **420 €**. |
| **Intentar recuperar de nuevo RB01** | La operación se rechaza sin modificar los datos. |
| **Buscar «rock» (CSV inicial)** | Se encuentra **Rockrider ST100**, aunque se utilicen mayúsculas distintas. |
| **Alta con valor 0, negativo o texto** | Se rechaza sin alterar el registro ni cerrar el menú. |
| **Guardar y volver a ejecutar** | Los códigos, estados y valoraciones se conservan correctamente. |

## 5. Condiciones técnicas

- Todas las llamadas a **print()** e **input()** deben permanecer **exclusivamente dentro de main()**. Los métodos y funciones auxiliares devuelven resultados, sin mostrar mensajes.
- Mantén la estructura del menú proporcionado. No utilices **while True** con **break**. Conserva **if __name__ == "__main__": main()**.
- Utiliza Python 3 sin bibliotecas externas ni bases de datos. La aplicación debe ejecutarse desde la carpeta del proyecto con **python main.py**.

> **Archivos que deben quedar completos al finalizar: main.py**, **modelos.py**, **persistencia.py** y **bicicletas.csv** con los cambios guardados.